BCC462
Inteligência Computacional

Otimização por Enxame de Partículas I

Profª Valéria

Introdução

- Otimização por Enxame de Partículas - *Particle Swarm Optimization* (PSO)
- Algoritmo de busca baseado em população
- Baseado na simulação do comportamento social de pássaros em um bando



Introdução

- Ideia inicial: simular graficamente a coreografia de um bando de pássaros
  - descobrir padrões que governam a habilidade dos pássaros voarem sincronicamente e, de repente, mudarem de direção e se reagruparem em uma formação ótima
  - o conceito evoluiu para um algoritmo simples e eficiente de otimização

Algoritmo PSO

- Cada indivíduo segue um princípio básico: simular o sucesso dos indivíduos vizinhos e seu próprio sucesso
- O comportamento coletivo que emerge desse comportamento simples é descobrir regiões ótimas em um espaço de busca *n*-dimensional

Algoritmo PSO

- O algoritmo mantém um grupo de partículas, onde cada partícula representa uma solução
- As partículas podem “voar” em um espaço de busca multidimensional, onde a posição de cada partícula é ajustada de acordo com sua própria experiência e de seus vizinhos

Posição da partícula

- Seja $x_i(t)$ a posição da partícula *i* no espaço de busca no tempo *t*; onde *t* denota passos de tempo discretos
- A posição da partícula muda pela adição da velocidade $v_i(t)$, para a posição atual:

$$x_i(t+1) = x_i(t) + v_i(t+1) \qquad (16.1)$$

- $x_i(0) \sim U(x_{min}, x_{max})$

Posição da partícula

- O vetor de velocidade é quem guia o processo de otimização e reflete ambos: conhecimento de experiência da partícula e informação da vizinhança da partícula
- O conhecimento de experiência é dito componente cognitivo, que é proporcional à distância da partícula de sua melhor posição encontrada
- A informação socialmente trocada é dita componente social da equação de velocidade

Algoritmos de PSO

- Há dois algoritmos principais que foram propostos, cuja diferença é o tamanho da vizinhança:
    - Global Best PSO
    - Local Best PSO

Global Best PSO

- *Global Best PSO* ou *gbest*
- A vizinhança de cada partícula é o grupo inteiro
- O componente social da atualização da velocidade da partícula reflete a informação obtida de todas as partículas no grupo
- Nesse caso, a informação social é a melhor posição encontrada pelo grupo, referida como $\hat{y}(t)$

Global Best PSO

- Para o gbest PSO, a velocidade da partícula *i* é calculada como:

$$v_{ij}(t+1) = v_{ij}(t) + c_1 r_{1j}(t)[y_{ij}(t) - x_{ij}(t)] + c_2 r_{2j}(t)[\hat{y}_j(t) - x_{ij}(t)]$$

- $v_{ij}(t)$ é a velocidade da partícula *i* na dimensão *j* = 1,...,n_x no tempo *t*
- $x_{ij}(t)$ é a posição da partícula *i* na dimensão *j* no tempo *t*
- c_1 e c_2 são constantes de aceleração positivas usadas para escalar a contribuição dos componentes social e cognitivo, respectivamente.

Global Best PSO

$$v_{ij}(t+1) = v_{ij}(t) + c_1 r_{1j}(t)[y_{ij}(t) - x_{ij}(t)] + c_2 r_{2j}(t)[\hat{y}_j(t) - x_{ij}(t)]$$

(16.2)

- $r_{1j}(t)$, $r_{2j}(t) \sim U(0,1)$ são valores aleatórios no intervalo [0,1], amostrados de uma distribuição uniforme
- y_i é a melhor posição pessoal da partícula desde a primeira iteração

Global Best PSO

- Considerando problemas de minimização, a posição pessoal no próximo passo de tempo, é calculada como:

$$\mathbf{y}_i(t+1) = \begin{cases} \mathbf{y}_i(t) & \text{if } f(\mathbf{x}_i(t+1)) \geq f(\mathbf{y}_i(t)) \\ \mathbf{x}_i(t+1) & \text{if } f(\mathbf{x}_i(t+1)) < f(\mathbf{y}_i(t)) \end{cases}$$

- f: $\mathbb{R}^{nx} \longrightarrow \mathbb{R}$ é a função de fitness
- A função de *fitness* mede o quão próximo a solução está da solução ótima
- A função de *fitness* quantifica a qualidade da partícula

Global Best PSO

- A melhor posição global, $\hat{y}(t)$, no tempo t, é definida como:

$$\hat{\mathbf{y}}(t) \in \{\mathbf{y}_0(t), \ldots, \mathbf{y}_{n_s}(t)\} | f(\hat{\mathbf{y}}(t)) = \min\{f(\mathbf{y}_0(t)), \ldots, f(\mathbf{y}_{n_s}(t))\}$$

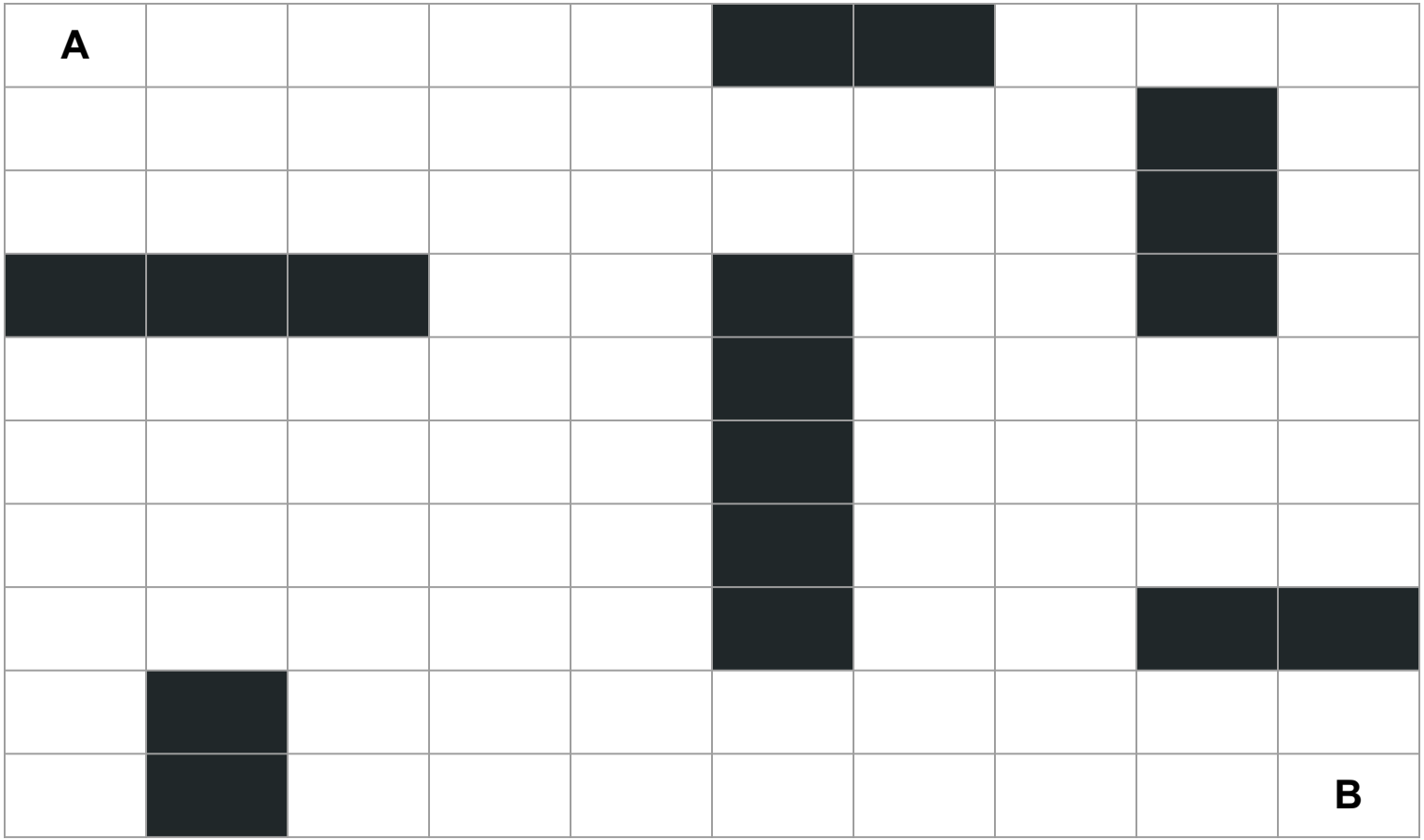
onde n_s é o número total de partículas no grupo.

Algorithm *gbest* PSO

Create and initialize an n_x-dimensional swarm;
repeat
    for *each particle* $i = 1, \ldots, n_s$ **do**
        //set the personal best position
        if $f(\mathbf{x}_i) < f(\mathbf{y}_i)$ **then**
            $\mathbf{y}_i = \mathbf{x}_i$;
        end
        //set the global best position **if** $f(\mathbf{y}_i) < f(\hat{\mathbf{y}})$ **then**
            $\hat{\mathbf{y}} = \mathbf{y}_i$;
        end
    end
    for *each particle* $i = 1, \ldots, n_s$ **do**
        update the velocity using equation (16.2);
        update the position using equation (16.1);
    end
until *stopping condition is true*;

Exemplo

- Ambiente:
  - Grade 10x10
  - Cada célula tem uma coordenada (x,y)
  - Ponto A (Início): Coordenada (0, 0)
  - Ponto B (Objetivo): Coordenada (9, 9)
  - Obstáculos: Células bloqueadas (ex: uma parede na coluna 5)


A
B

Função de fitness

- *f* = distância euclidiana até o objetivo + penalidade por colisão

onde a Distância Euclidiana entre a posição atual (x_1, y_1) e o alvo (x_2, y_2) é:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Comportamento do enxame

Cada partícula decide seu próximo passo baseada em três coisas:

1. Inércia: "Vou continuar para onde eu já estava indo."
2. Memória Individual (p_{best}): "Vou voltar para o lugar mais perto do objetivo que eu já encontrei."
3. Conhecimento Social (G_{best}): "Vou seguir o caminho que o meu colega descobriu ser o melhor de todos."

Local Best PSO

- No *local best PSO*, ou *lbest*, vizinhanças menores são definidas para cada partícula
- O componente social reflete a informação trocada com o vizinho da partícula, refletindo conhecimento local do ambiente
- Em relação à equação de velocidade, a contribuição social para a velocidade da partícula é proporcional à distância entre uma partícula e a melhor posição encontrada pela vizinhança das partículas

Local Best PSO

- A velocidade é calculada como:

$$v_{ij}(t+1) = v_{ij}(t) + c_1 r_{1j}(t)[y_{ij}(t) - x_{ij}(t)] + c_2 r_{2j}(t)[\hat{y}_{ij}(t) - x_{ij}(t)] \quad (16.6)$$

onde $\hat{y}_{ij}$ é a melhor posição encontrada pela vizinhança da partícula *i* na dimensão *j*

Local Best PSO

- A melhor posição local da partícula, $\hat{y}_i$, é definida como:

$$\hat{\mathbf{y}}_i(t+1) \in \{\mathcal{N}_i | f(\hat{\mathbf{y}}_i(t+1)) = \min\{f(\mathbf{x})\}, \quad \forall \mathbf{x} \in \mathcal{N}_i\}$$

com a vizinhança definida como:

$$\mathcal{N}_i = \{\mathbf{y}_{i-n_{\mathcal{N}_i}}(t), \mathbf{y}_{i-n_{\mathcal{N}_i}+1}(t), \ldots, \mathbf{y}_{i-1}(t), \mathbf{y}_i(t), \mathbf{y}_{i+1}(t), \ldots, \mathbf{y}_{i+n_{\mathcal{N}_i}}(t)\}$$

Algorithm *lbest* PSO

Create and initialize an n_x-dimensional swarm;
repeat
    for *each particle* $i = 1, \ldots, n_s$ **do**
        //set the personal best position
        if $f(\mathbf{x}_i) < f(\mathbf{y}_i)$ **then**
            $\mathbf{y}_i = \mathbf{x}_i$;
        end
        //set the neighborhood best position
        if $f(\mathbf{y}_i) < f(\hat{\mathbf{y}}_i)$ **then**
            $\hat{\mathbf{y}} = \mathbf{y}_i$;
        end
    end
    for *each particle* $i = 1, \ldots, n_s$ **do**
        update the velocity using equation (16.6);
        update the position using equation (16.1);
    end
until *stopping condition is true*;

gbest x lbest PSO

- As duas versões são similares no sentido que a atualização do componente social de velocidade causa o movimento para a melhor partícula global (*lbest*: sobreposição de vizinhanças)

gbest x lbest PSO

- Há duas diferenças principais em relação às características de convergência:
    - Devido à maior interconectividade do gbest PSO, ele converge mais rápido que o lbest. Entretanto, essa rapidez leva ao custo de menos diversidade que o lbest
    - Como é lbest tem maior diversidade, ele é menos suscetível a cair em um mínimo local

Componentes de velocidade

- O cálculo da velocidade nos dois algoritmos consiste de três termos:
    - Velocidade anterior
    - Componente cognitivo
    - Componente social

Componentes de velocidade

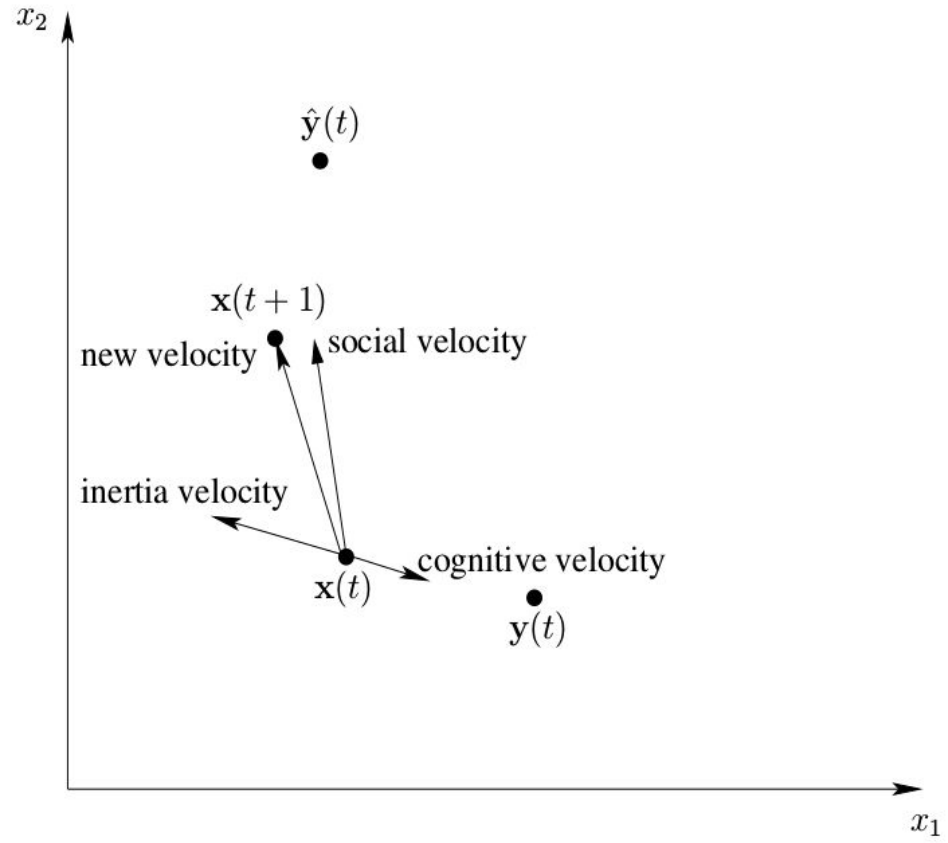
- **Velocidade anterior** $v_i(t)$:
  - serve como uma memória da direção do movimento passado;
  - evita que a partícula mude bruscamente de direção e a guia para a direção atual.
  - também chamado de componente de inércia.

Componentes de velocidade

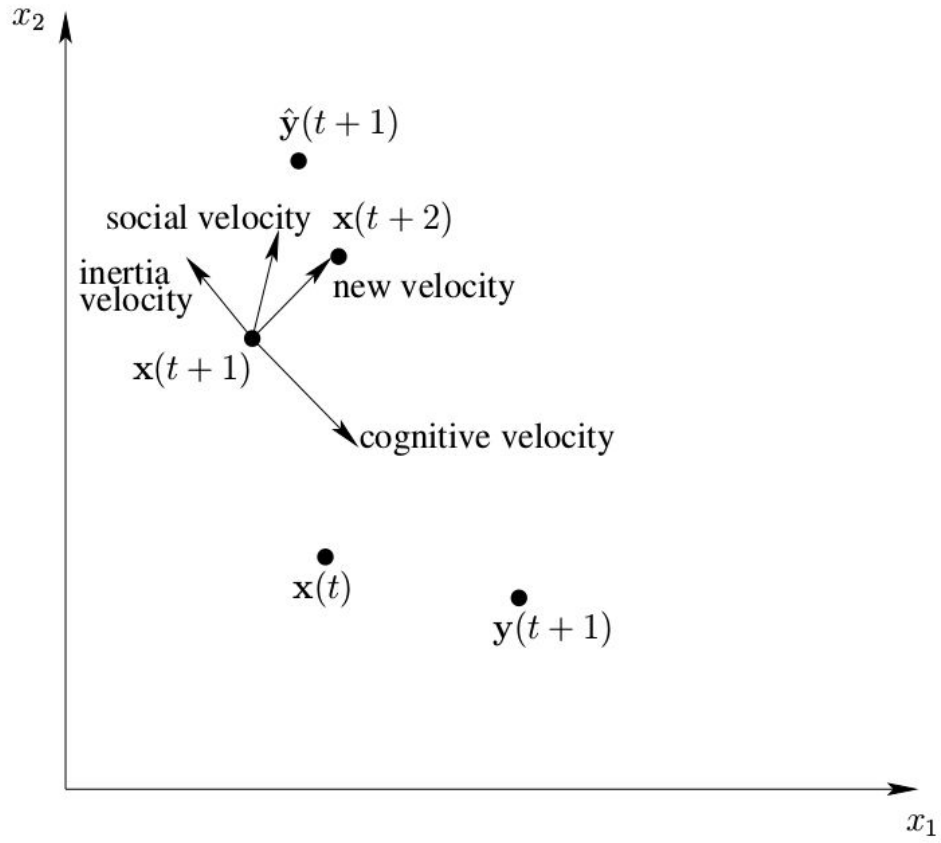
- **Componente cognitivo** $c_1 r_1 (y_i - x_i)$:
  - quantifica o desempenho da partícula *i* em relação ao desempenho passado;
  - permite que as partículas recuem para suas melhores posições;
  - semelhante à tendência dos indivíduos retornarem a situações mais satisfatórias do passado.

Componentes de velocidade

- **Componente social** $c_2 r_2(\hat{y}-x_i)$, no *gbest*, ou $c_2 r_2(\hat{y}_i-x_i)$, no *lbest*:
  - quantifica o desempenho da partícula *i* relativa ao grupo de partículas ou vizinhos;
  - cada partícula também volta à melhor posição encontrada pela vizinhança da partícula.



(a) Time Step t



(b) Time Step $t+1$

Aspectos do algoritmo

- Inicialização
- Critério de parada
- Definição de termos de iteração
- Função de avaliação

Inicialização

- As constantes de aceleração, c1 e c2
- As velocidades iniciais
- As posições das partículas
- Melhores posições individuais
- Tamanho da vizinhança - lbest PSO

Inicialização

- As posições das partículas são inicializadas para cobrir uniformemente o espaço de busca;
- A eficiência do PSO é influenciada pela diversidade inicial do enxame:
  - quanto do espaço de busca é coberto;
  - quão bem as partículas são distribuídas no espaço de busca.

Inicialização

- As velocidades iniciais podem ser inicializadas com 0;
  - Inicialização aleatória de vetores de posição já garante posições e direções de movimento aleatórias.
- Se as velocidades iniciais também tiverem valores aleatórios, eles não devem ser grandes
  - Velocidades iniciais grandes levam a grandes atualizações de posições
    - Maior risco que extrapolar os limites do espaço de busca;
    - Mais iterações para convergir.

Inicialização

- A melhor posição individual de cada partícula é a posição da partícula no tempo t=0:

$$y_i(0) = x_i(0)$$

Critério de parada

- Dois aspectos devem ser considerados:
  - não pode causar convergência prematura, onde soluções sub-ótimas são obtidas;
  - deve evitar a sobreamostragem da função de aptidão.

  Se uma condição de parada exigir cálculos frequentes da função de aptidão, a complexidade computacional do processo de busca pode aumentar significativamente.

Inicialização

- Critérios de parada:
  - Atingir um número máximo de iterações;
  - Uma solução aceitável foi encontrada;
  - Nenhuma melhora é observada após um determinado número de iterações;
  - Raio normalizado do enxame é próximo a zero:
  - Gradiente da função objetivo é próximo a zero.